Today 10:07 AM

Lab 4: VibeCoding Objective Use Vibe Coding tools to fix the broken code and add features to the same. Learn how to prompt to get the job done in under three to four attempts. Clone a repo assigned to you, make changes and push to your repo in your personal account. THIS IS AN INDIVIDUAL ASSIGNMENT. WORK INDIVIDUALLY!! Duration 60 minutes Software Requirements ● Access to Vibe Coding tool. (ChatGPT, Gemini, Claude etc) Instructions: 1. Go to the repo assigned to you 2. Go through the ReadMe file to understand the deliverables 3. Clone or Fork the repo 4. Run the Python based code (A game/ A Puzzle) 5. Record 10 seconds of video before making any changes. 6. Write prompts and fix the broken code 7. Write prompts to add all the features listed in the ReadMe file of the repo one by one. 8. Record another 10 seconds of video again after making all the changes. 9. Push the code to your repo. 10. Do not raise PR to the main repo (SETAPESU26) Deliverables: Upload the following to your Lab repo under Lab-4 folder, a. Videos (before and after) b. Updated Code c. Chat history exported as a doc/pdf github link
https://github.com/SETAPESU26/29_digDug help me complete this guide me step by step

Worked for 13s

PS C:\Users\monik\OneDrive\Documents\Desktop> git clone
https://github.com/Monikapm19/lab_4.git
Cloning into 'lab_4'...
fatal: unable to access '
https://github.com/Monikapm19/lab_4.git/': Could not resolve host: github.com what is this

PS C:\Users\monik\OneDrive\Documents\Desktop> ping github.com

Pinging github.com [20.207.73.82] with 32 bytes of data:
Reply from 20.207.73.82: bytes=32 time=1620ms TTL=113
Reply from 20.207.73.82: bytes=32 time=112ms TTL=113
Reply from 20.207.73.82: bytes=32 time=642ms TTL=113
Reply from 20.207.73.82: bytes=32 time=1334ms TTL=113

Ping statistics for 20.207.73.82:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 112ms, Maximum = 1620ms, Average = 927ms

PS C:\Users\monik\OneDrive\Documents\Desktop\lab_4> python -m pip install pygame
Defaulting to user installation because normal site-packages is not writeable

tep 4 — Fix the bug and add all three features

The README specifies these four coding tasks. <Cite refs= {["turn240436view0"]}/>

Task	Required change
1	Fix BFS pathfinding so enemies take the shortest route through cleared tunnels
2	Implement `dirt_color(row)` to give dirt distinct colors by depth
3	Implement `on_enemy_popped(enemy, score)` for a particle burst or depth-based bonus points
4	

Implement `enemy_speed_multiplier(level)` so enemies become faster in later levels
After recording the before-video, we'll use AI prompts to implement these tasks one by one. We'll inspect the actual code, review each change, and test it before moving to the next feature.
give me step by step what to do

Worked for 10s

you give me code alos what to edit

Worked for 20s

you give me code alos what to edit task by rask

Worked for 15s

```
import random
from collections import deque

import pygame

TILE, COLS, ROWS = 32, 20, 14
WIDTH, HEIGHT = COLS * TILE, ROWS * TILE + 36
DIRS = {pygame.K_UP: (-1, 0), pygame.K_DOWN: (1, 0),
pygame.K_LEFT: (0, -1), pygame.K_RIGHT: (0, 1)}
MOVE_DELAY, ENEMY_DELAY, PUMP_RANGE, DEFLATE_AFTER =
0.11, 0.35, 3, 1.5


def dirt_color(row):
    """Return an (r, g, b) colour for dirt in the given row, or None for
the default gradient."""
    pass


def on_enemy_popped(enemy, score):
    """Called when an enemy is popped; add particles, bonus
points, or a colour flash here."""
    pass
```

```
def enemy_speed_multiplier(level):
    """Return a speed multiplier for enemies at the given level, or
None for the default speed."""
    pass


def in_bounds(r, c):
    return 0 <= r < ROWS and 0 <= c < COLS


def bfs_path(grid, start, goal):
    queue, parents = deque([start]), {start: None}
    while queue:
        cell = queue.pop()
        if cell == goal:
            break
        for dr, dc in DIRS.values():
            nxt = (cell[0] + dr, cell[1] + dc)
            if in_bounds(*nxt) and not grid[nxt[0]][nxt[1]] and nxt not in
parents:
                parents[nxt] = cell
                queue.append(nxt)
    if goal not in parents:
        return []
    path, cell = [], goal
    while cell != start:
        path.append(cell)
        cell = parents[cell]
    return path[::-1]


class Enemy:
    def __init__(self, cell):
        self.cell = cell
        self.stage = 0
        self.last_pump = 0.0
        self.step_timer = random.uniform(0, ENEMY_DELAY)
        self.ghost_timer = random.uniform(6, 10)
        self.ghost = False
```

```
    @property
    def locked(self):
        return self.stage > 0


class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.level, self.score, self.lives, self.state = 1, 0, 3, "play"
        self.start_level()

    def start_level(self):
        self.grid = [[r > 0 for _ in range(COLS)] for r in range(ROWS)]
        self.player = (0, COLS // 2)
        self.facing = (0, 1)
        self.move_timer = 0.0
        self.time = 0.0
        self.pump_target = None
        self.enemies = []
        for _ in range(3 + self.level):
            r, c = random.randint(4, ROWS - 2), random.randint(1,
COLS - 3)
            for dc in range(3):
                self.grid[r][c + dc] = False
            self.enemies.append(Enemy((r, c + random.randint(0, 2))))

    def dig(self, cell):
        self.grid[cell[0]][cell[1]] = False

    def move_player(self, direction):
        self.facing = direction
        nr, nc = self.player[0] + direction[0], self.player[1] +
direction[1]
        if in_bounds(nr, nc):
            self.player = (nr, nc)
            self.dig(self.player)
```

```
    def find_pump_target(self):
        for step in range(1, PUMP_RANGE + 1):
            r, c = self.player[0] + self.facing[0] * step, self.player[1] +
self.facing[1] * step
            if not in_bounds(r, c) or self.grid[r][c]:
                return None
            for enemy in self.enemies:
                if enemy.cell == (r, c):
                    return enemy
        return None

    def pump(self):
        enemy = self.find_pump_target()
        self.pump_target = enemy
        if enemy is None:
            return
        enemy.stage += 1
        enemy.last_pump = self.time
        if enemy.stage >= 4:
            self.enemies.remove(enemy)
            self.score += 200 + 100 * (enemy.cell[0] // 4)
            self.pump_target = None
            on_enemy_popped(enemy, self.score)
            if not self.enemies:
                self.level += 1
                self.start_level()

    def hurt(self):
        self.lives -= 1
        self.player = (0, COLS // 2)
        if self.lives <= 0:
            self.state = "lose"

    def update_enemy(self, enemy, dt):
        if enemy.locked:
            if self.time - enemy.last_pump > DEFLATE_AFTER:
                enemy.stage -= 1
                enemy.last_pump = self.time
            return
        enemy.ghost_timer -= dt
```

```
            if enemy.ghost_timer <= 0:
                enemy.ghost = not enemy.ghost
                enemy.ghost_timer = 4.0 if enemy.ghost else
random.uniform(6, 10)
                if not enemy.ghost:
                    self.grid[enemy.cell[0]][enemy.cell[1]] = False
            enemy.step_timer -= dt
            if enemy.step_timer > 0:
                return
            enemy.step_timer = ENEMY_DELAY * (1.6 if enemy.ghost else
1) / (enemy_speed_multiplier(self.level) or 1)
            if enemy.ghost:
                dr, dc = self.player[0] - enemy.cell[0], self.player[1] -
enemy.cell[1]
                if abs(dr) >= abs(dc) and dr:
                    enemy.cell = (enemy.cell[0] + (1 if dr > 0 else -1),
enemy.cell[1])
                elif dc:
                    enemy.cell = (enemy.cell[0], enemy.cell[1] + (1 if dc > 0
else -1))
                return
            path = bfs_path(self.grid, enemy.cell, self.player)
            if path:
                enemy.cell = path[0]

        def update(self, dt, keys):
            if self.state != "play":
                return
            self.time += dt
            self.move_timer -= dt
            if self.move_timer <= 0:
                for key, direction in DIRS.items():
                    if keys[key]:
                        self.move_player(direction)
                        self.move_timer = MOVE_DELAY
                        break
            for enemy in self.enemies[:]:
                self.update_enemy(enemy, dt)
                if enemy.cell == self.player and not enemy.locked:
                    self.hurt()
```

```
                break
        if self.pump_target and (self.pump_target not in self.enemies
or not self.pump_target.locked):
            self.pump_target = None

    def draw(self, screen):
        screen.fill((30, 30, 60))
        for r in range(ROWS):
            for c in range(COLS):
                rect = pygame.Rect(c * TILE, r * TILE, TILE, TILE)
                if r == 0:
                    pygame.draw.rect(screen, (90, 170, 230), rect)
                elif self.grid[r][c]:
                    shade = dirt_color(r) or (170 - r * 6, 110 - r * 4, 60)
                    pygame.draw.rect(screen, shade, rect)
                else:
                    pygame.draw.rect(screen, (20, 12, 10), rect)
        for enemy in self.enemies:
            x, y = enemy.cell[1] * TILE + TILE // 2, enemy.cell[0] * TILE +
TILE // 2
            radius = 10 + enemy.stage * 4
            color = (230, 230, 240) if enemy.ghost else (230, 70, 70)
            pygame.draw.circle(screen, color, (x, y), radius)
            pygame.draw.circle(screen, (255, 255, 255), (x - 4, y - 3), 3)
            pygame.draw.circle(screen, (255, 255, 255), (x + 4, y - 3), 3)
        px, py = self.player[1] * TILE + TILE // 2, self.player[0] * TILE +
TILE // 2
        pygame.draw.circle(screen, (250, 250, 250), (px, py), 12)
        pygame.draw.rect(screen, (60, 120, 230), (px - 8, py - 12, 16,
8))
        if self.pump_target:
            tx, ty = self.pump_target.cell[1] * TILE + TILE // 2,
self.pump_target.cell[0] * TILE + TILE // 2
            pygame.draw.line(screen, (255, 255, 120), (px, py), (tx, ty), 3)
        hud = self.font.render(f"Score {self.score}  Lives {self.lives}
Level {self.level}  R = reset", True, (240, 240, 240))
        screen.blit(hud, (10, ROWS * TILE + 8))
        if self.state == "lose":
            label = self.font.render("GAME OVER - Press R", True, (255,
255, 120))
```

```
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT
// 2)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Dig Dug")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_SPACE and game.state == "play":
                game.pump()
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:
                game.reset()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
    pygame.quit()


if __name__ == "__main__":
    main()
```

i am done puching also

how to make chat_history pdf